ADL - Challenge Report
COLABorative

Pietro Beghetto, Alessandro Ghiotto, Alessandro Avallone

pietrobeghetto, alessandroghiotto, zander121

279029, 303035, 286399

May 24, 2026

1 Introduction

The ADL 2025–2026 challenge targets *industrial visual anomaly detection*: given five product classes photographed from five views each, the task is to detect and localise defects at the pixel level. Our final submission is a two-branch system: an unsupervised detector built on PatchCore [10] with a DINOv2 [8] backbone, and a supervised U-Net [9] trained on synthetic defects.

2 Development Process

2.1 Problem Analysis

- **Label scarcity:** We were provided with only a few labelled anomalies for each class, naturally pushing us towards an unsupervised or weakly supervised solution.
- **Inconsistent views:** The view IDs were not correlated with a specific view angle.
- **Anomaly descriptions:** Each ground truth mask was accompanied by a brief description of the defect it was highlighting.

2.2 First attempts: reconstruction and student–teacher

We started with pure reconstruction baselines (vanilla and denoising autoencoders). The reconstruction loss was too coarse: the network learned to reconstruct anomalies almost as well as normal regions, and the anomaly heatmaps were contaminated by noise outside the object. The student–teacher paradigm [2], with a Wide-ResNet-50-2 teacher and a ResNet-18 student trained to mimic its features on normal data, gave better localisation but the score map still leaked heavily onto the background and on cluttered fixtures, which strongly suggested that **background removal** had to come first.

2.3 BiRefNet preprocessing

We therefore preprocessed the whole dataset with BiRefNet [12]: a high-resolution segmenter that produces a soft foreground mask per image. We binarise it at 0.5, and composite the foreground over a constant black fill. Ground-truth anomaly masks are not touched. All subsequent models use this cleaned dataset, which removes a large source of false positives along the object's silhouette and in the background.

2.4 Pose clustering

Since the filename suffix `viewN` does *not* consistently encode the same camera angle across different physical objects, to recover pose-coherent groups we mean-pool DINOv2 patch tokens into a 768-d descriptor per image, L_2-normalise, and run k-means per class sweeping $K \in [2, 8]$ and picking K by silhouette score. The resulting assignments are reused downstream to constrain CutPaste sampling and for FiLM conditioning.

2.5 PatchCore with DINOv2

PatchCore became our first competitive detector. We extract dense patch tokens from a frozen `vit_large_patch14_reg4_dinov2` at 518×518, concatenating layers $\{7, 12, 16, 20\}$ to mix mid-level texture and high-level semantics, then project the resulting $4 \cdot 1024$-d features down to 512 components with a randomised-SVD PCA. A greedy coreset of $\leq 10^5$ anchors is built per class and stored as a tensor; at inference time the anomaly score of each patch is the mean Euclidean distance to its $k = 9$ nearest coreset entries. The patch grid is smoothed with a Gaussian ($\sigma = 1$) and bilinearly upsampled to image resolution.

Replacing the ImageNet-pretrained Wide-ResNet of the original formulation [10] with DINOv2 features gave a significant boost to pixel AP: the ViT representations are far less sensitive to global appearance variations and emphasise local structure, which is what most industrial defect amount to.

2.6 Supervised U-Net on synthetic anomalies

PatchCore alone leaves a clear failure mode: subtle, low-contrast defects whose features are too close to the normal manifold. A complementary supervised branch trained with explicit labels is the natural fix, but only a few dozen real anomalies per class are available. We therefore turned the few labelled anomalies and a large pool of normal images into a synthetic segmentation training set.

Synthetic anomaly generation. The `ShapeTextureBank` decouples a defect's *shape* from its *texture*: shapes are sampled from a mixture of real masks (30%), elastically warped real masks (25%), cut-mixed pairs of real masks (10%), Perlin-noise blobs (15%), Bézier strokes (15%) and random walks (5%); textures are bbox-cropped pixels from real defects. At each `__getitem__` call on a normal image, with probability p_{cutpaste}=0.3 a defect is pasted via CutPaste [5] with geometric and photometric jitter (random flip, rotation, mask dilation/erosion, brightness/contrast, feathered alpha-blending), and with probability p_{multi}=0.3 a second crop is pasted on top. With probability p_{synth}=0.4 a purely procedural defect is drawn instead: scratches, stains, indents, cracks, fragments, holes, fuzz; the motif list per class is derived by keyword-matching the anomaly descriptions CSV. **Pose-aware sampling** restricts every paste to a source crop that shares the destination's pose cluster; this is important because pasting a top-down defect onto a side view produces unrealistic supervision. Anomalous training images, which lack a pose label, are assigned to the nearest labelled good image of the same class via 32×32 grayscale L_2 nearest neighbour.

Architecture. The network is a U-Net with a `resnet34` ImageNet-pretrained encoder, four upsampling stages with **attention gates** [7] on every skip connection, and a *view-id* FiLM conditioning that lets a single per-class model specialise its decoder per camera angle. Training uses a 0.5:0.5 BCE+Dice loss, masked to the BiRefNet foreground.

Inference. Test-time augmentation (identity + horizontal + vertical flip) is averaged, the BiRefNet foreground is re-applied at test time, and the five views of a physical sample are *co-amplified*: if at least one view confidently fires, the others are scaled by a factor proportional to the maximum confidence.

2.7 Ensemble

PatchCore and the U-Net make complementary mistakes. PatchCore is sharp on out-of-distribution patches but produces a ring artefact along the object contour and noisy specks inside textured regions; the U-Net is clean and shape-aware but conservative on rare motifs. Before fusion we therefore (i) morphologically open the PatchCore map with a r=3 kernel and drop connected components smaller than 50 pixels above the p_{99} threshold, and (ii) normalise both maps per class using the *excess-above-*

p_{99} statistic of good-image scores so that the noise floor maps to 0. The fused map is then

$$s_{\text{fused}} = \alpha\, s_{\text{U-Net}} + (1-\alpha)\, s_{\text{PC}}, \tag{1}$$

with the per-class weight $\alpha \in \{0, 0.05, \ldots, 1\}$ chosen by grid search to maximise pooled pixel-AP on the labelled training anomalies of that class. The weighted sum was the most robust on validation.

Validation protocol. To select α and compare strategies we construct a held-out proxy split entirely from the labelled training data. For each class, 20% of the normal images are withheld as *val-normals*; the remaining 80% are used to rebuild a class-specific PatchCore bank with the same coreset and PCA pipeline, and to re-estimate the p_{99} calibration constant. All labelled training anomalies are added to the validation pool with their pixel masks. Both PatchCore and the U-Net are then scored on every image in the pool. Per-class α values are swept independently and the resulting global pixel-AP is compared against the single best-α baseline. The same subsample-then-verify protocol was used to evaluate the alternative fusion rules (`max`, `geometric_mean`, `cascaded`, `two_tier`) and the PatchCore power transform s_{PC}^{γ} for $\gamma \in \{0.5, 0.75, 1.25, 1.5, 2.0, 2.5, 3.0\}$.

2.8 Other approaches we tried shallowly

We additionally ran controlled experiments with DRAEM [11], EfficientAD [1], RD4AD [3], SimpleNet [6] and the few-shot WinCLIP+ [4] detector. None of them was competitive with the PatchCore+U-Net pair on this dataset under our time budget — partly because the small per-class regime penalises the models that need long training (DRAEM, EfficientAD, SimpleNet), and partly because few-shot prompting (WinCLIP+) lacks the fine spatial resolution required by the pixel-AP metric. Their code is kept in `final/extra/` for reference.

3 Final Architecture

Putting the pieces together, the final pipeline is:

1. **BiRefNet** background removal applied once to the whole dataset (preprocessing).
2. **Pose clustering** on DINOv2 global descriptors, saved per image (preprocessing).
3. **PatchCore** with DINOv2 features, PCA-512 and a 10^5-anchor coreset, post-processed with morphological opening and small-blob removal.
4. **U-Net** (resnet34 encoder, attention gates, view-id FiLM) trained on synthetic anomalies generated by the `ShapeTextureBank` with pose-aware CutPaste, BCE + Dice loss masked to the foreground, TTA + multi-view amplification at inference.
5. Per-class **weighted-sum fusion** (Eq. 1) with α selected on labelled validation, followed by a final σ=0.9 Gaussian smoothing.

4 Discussion

Some of the decisions disproportionately moved the leaderboard: **(i)** replacing ImageNet features with DINOv2 inside PatchCore, **(ii)** masking out the background with BiRefNet end-to-end, and **(iii)** calibrating raw PatchCore scores by dividing each test heatmap by the 99th percentile of per-image maximum scores on normal training images, mapping distances to a consistent $[0, 1]$ range across classes. **(i)** and **(ii)** target the same failure mode — the model spending capacity on pixels that the metric does not reward, while **(iii)** targets a different failure mode — raw k-NN distances having incomparable magnitudes across classes. The ensemble itself adds a more modest but consistent gain on top, mostly on classes where PatchCore is dominant and the U-Net acts as a precision filter, and vice versa. The main remaining weakness is on defects that are both small and low-contrast: PatchCore underweights them because their features are close to the normal manifold, and the U-Net underweights them because the synthetic distribution rarely matches them. A natural next step would be to feed PatchCore's confidence as an additional input channel to the U-Net so that the supervised branch can learn where to focus its capacity. Aggiungerei forse una nota sul fatto che in alcune classi performava molto peggio rispetto ad altre (pistacchio la peggiore, ingranaggi sorprendentemente male).

References

[1] K. Batzner, L. Heckler, and R. König. EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In *WACV*, 2024.

[2] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger. Uninformed students: Student–teacher anomaly detection with discriminative latent embeddings. In *CVPR*, 2020.

[3] H. Deng and X. Li. Anomaly detection via reverse distillation from one-class embedding. In *CVPR*, 2022.

[4] J. Jeong, Y. Zou, T. Kim, D. Zhang, A. Ravichandran, and O. Dabeer. WinCLIP: Zero-/few-shot anomaly classification and segmentation. In *CVPR*, 2023.

[5] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister. CutPaste: Self-supervised learning for anomaly detection and localization. In *CVPR*, 2021.

[6] Z. Liu, Y. Zhou, Y. Xu, and Z. Wang. SimpleNet: A simple network for image anomaly detection and localization. In *CVPR*, 2023.

[7] O. Oktay, J. Schlemper, L. Le Folgoc, M. Lee, M. Heinrich, K. Misawa, K. Mori, S. McDonagh, N. Y. Hammerla, B. Kainz, et al. Attention u-net: Learning where to look for the pancreas. *arXiv:1804.03999*, 2018.

[8] M. Oquab, T. Darcet, T. Moutakanni, et al. DINOv2: Learning robust visual features without supervision. *TMLR*, 2024.

[9] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.

[10] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler. Towards total recall in industrial anomaly detection. In *CVPR*, 2022.

[11] V. Zavrtanik, M. Kristan, and D. Skočaj. DRAEM: A discriminatively trained reconstruction embedding for surface anomaly detection. In *ICCV*, 2021.

[12] P. Zheng, D. Gao, D.-P. Fan, L. Liu, J. Laaksonen, W. Ouyang, and N. Sebe. Bilateral reference for high-resolution dichotomous image segmentation. In *CAAI Artificial Intelligence Research*, 2024.